

```

        barVal = ::val;           // 正确：使用一个全局对象
        barVal = si;             // 正确：使用一个静态局部对象
        locVal = b;              // 正确：使用一个枚举成员
    }
};
// ...
}

```

常规的访问保护规则对局部类同样适用

外层函数对局部类的私有成员没有任何访问特权。当然，局部类可以将外层函数声明为友元；或者更常见的情况是局部类将其成员声明成公有的。在程序中有权访问局部类的代码非常有限。局部类已经封装在函数作用域中，通过信息隐藏进一步封装就显得没什么必要了。

局部类中的名字查找

局部类内部的名字查找次序与其他类相似。在声明类的成员时，必须先确保用到的名字位于作用域中，然后再使用该名字。定义成员时用到的名字可以出现在类的任意位置。如果某个名字不是局部类的成员，则继续在外层函数作用域中查找；如果还没有找到，则在外层函数所在的作用域中查找。

854

嵌套的局部类

可以在局部类的内部再嵌套一个类。此时，嵌套类的定义可以出现在局部类之外。不过，嵌套类必须定义在与局部类相同的作用域中。

```

void foo()
{
    class Bar {
    public:
        // ...
        class Nested; // 声明 Nested 类
    };
    // 定义 Nested 类
    class Bar::Nested {
        // ...
    };
}

```

和往常一样，当我们在类的外部定义成员时，必须指明该成员所属的作用域。因此在上面的例子中，`Bar::Nested` 的意思是 `Nested` 是定义在 `Bar` 的作用域内的一个类。

局部类内的嵌套类也是一个局部类，必须遵循局部类的各种规定。嵌套类的所有成员都必须定义在嵌套类内部。

19.8 固有的不可移植的特性

为了支持低层编程，C++定义了一些固有的不可移植（nonportable）的特性。所谓不可移植的特性是指因机器而异的特性，当我们将含有不可移植特性的程序从一台机器转移到另一台机器上时，通常需要重新编写该程序。算术类型的大小在不同机器上不一样（参见 2.1.1 节，第 30 页），这是我们使用过的不可移植特性的一个典型示例。

本节将介绍 C++ 从 C 语言继承而来的另外两种不可移植的特性：位域和 volatile 限定符。此外，我们还将介绍链接指示，它是 C++ 新增的一种不可移植的特性。

19.8.1 位域

类可以将其（非静态）数据成员定义成位域（bit-field），在一个位域中含有一定数量的二进制位。当一个程序需要向其他程序或硬件设备传递二进制数据时，通常会用到位域。

Note

位域在内存中的布局是与机器相关的。

855

位域的类型必须是整型或枚举类型（参见 19.3 节，第 736 页）。因为带符号位域的行为是由具体实现确定的，所以在通常情况下我们使用无符号类型保存一个位域。位域的声明形式是在成员名字之后紧跟一个冒号以及一个常量表达式，该表达式用于指定成员所占的二进制位数：

```
typedef unsigned int Bit;
class File {
    Bit mode: 2;                // mode 占 2 位
    Bit modified: 1;            // modified 占 1 位
    Bit prot_owner: 3;          // prot_owner 占 3 位
    Bit prot_group: 3;          // prot_group 占 3 位
    Bit prot_world: 3;          // prot_world 占 3 位
    // File 的操作和数据成员
public:
    // 文件类型以八进制的形式表示，参见 2.1.3 节（第 35 页）
    enum modes { READ = 01, WRITE = 02, EXECUTE = 03 };
    File &open(modes);
    void close();
    void write();
    bool isRead() const;
    void setWrite();
};
```

mode 位域占 2 个二进制位，modified 只占 1 个，其他成员则各占 3 个。如果可能的话，在类的内部连续定义的位域压缩在同一整数的相邻位，从而提供存储压缩。例如在之前的声明中，五个位域可能会存储在同一个 unsigned int 中。这些二进制位是否能压缩到一个整数中以及如何压缩是与机器相关的。

取地址运算符（&）不能作用于位域，因此任何指针都无法指向类的位域。



通常情况下最好将位域设为无符号类型，存储在带符号类型中的位域的行为将因具体实现而定。

使用位域

访问位域的方式与访问类的其他数据成员的方式非常相似：

```
void File::write()
{
    modified = 1;
```

```

    // ...
}
void File::close()
{
    if (modified)
        // ..... 保存内容
}

```

通常使用内置的位运算符（参见 4.8 节，第 136 页）操作超过 1 位的位域：

856

```

File &File::open(File::modes m)
{
    mode |= READ;           // 按默认方式设置 READ
    // 其他处理
    if (m & WRITE)          // 如果打开了 READ 和 WRITE
        // 按照读/写方式打开文件
    return *this;
}

```

如果一个类定义了位域成员，则它通常也会定义一组内联的成员函数以检验或设置位域的值：

```

inline bool File::isRead() const { return mode & READ; }
inline void File::setWrite() { mode |= WRITE; }

```

19.8.2 volatile 限定符



volatile 的确切含义与机器有关，只能通过阅读编译器文档来理解。要想让使用了 **volatile** 的程序在移植到新机器或新编译器后仍然有效，通常需要对程序进行某些改变。

直接处理硬件的程序常常包含这样的数据元素，它们的值由程序直接控制之外的过程控制。例如，程序可能包含一个由系统时钟定时更新的变量。当对象的值可能在程序的控制或检测之外被改变时，应该将该对象声明为 **volatile**。关键字 **volatile** 告诉编译器不应应对这样的对象进行优化。

volatile 限定符的用法和 **const** 很相似，它起到对类型额外修饰的作用：

```

volatile int display_register;    // 该 int 值可能发生改变
volatile Task *curr_task;        // curr_task 指向一个 volatile 对象
volatile int iax[max_size];      // iax 的每个元素都是 volatile
volatile Screen bitmapBuf;       // bitmapBuf 的每个成员都是 volatile

```

const 和 **volatile** 限定符互相没什么影响，某种类型可能既是 **const** 的也是 **volatile** 的，此时它同时具有二者的属性。

就像一个类可以定义 **const** 成员函数一样，它也可以将成员函数定义成 **volatile** 的。只有 **volatile** 的成员函数才能被 **volatile** 的对象调用。

2.4.2 节（第 56 页）描述了 **const** 限定符和指针的相互作用，在 **volatile** 限定符和指针之间也存在类似的关系。我们可以声明 **volatile** 指针、指向 **volatile** 对象的指针以及指向 **volatile** 对象的 **volatile** 指针：

```

volatile int v;                // v 是一个 volatile int

```

857

```

int *volatile vip;    // vip 是一个 volatile 指针，它指向 int
volatile int *ivp;    // ivp 是一个指针，它指向一个 volatile int
// vivp 是一个 volatile 指针，它指向一个 volatile int
volatile int *volatile vivp;

int *ip = &v;         // 错误：必须使用指向 volatile 的指针
ivp = &v;             // 正确：ivp 是一个指向 volatile 的指针
vivp = &v;            // 正确：vivp 是一个指向 volatile 的 volatile 指针

```

和 `const` 一样，我们只能将一个 `volatile` 对象的地址（或者拷贝一个指向 `volatile` 类型的指针）赋给一个指向 `volatile` 的指针。同时，只有当某个引用是 `volatile` 的时，我们才能使用一个 `volatile` 对象初始化该引用。

合成的拷贝对 `volatile` 对象无效

`const` 和 `volatile` 的一个重要区别是我们不能使用合成的拷贝/移动构造函数及赋值运算符初始化 `volatile` 对象或从 `volatile` 对象赋值。合成的成员接受的形参类型是（非 `volatile`）常量引用，显然我们不能把一个非 `volatile` 引用绑定到一个 `volatile` 对象上。

如果一个类希望拷贝、移动或赋值它的 `volatile` 对象，则该类必须自定义拷贝或移动操作。例如，我们可以将形参类型指定为 `const volatile` 引用，这样我们就能利用任意类型的 `Foo` 进行拷贝或赋值操作了：

```

class Foo {
public:
    Foo(const volatile Foo&); // 从一个 volatile 对象进行拷贝
    // 将一个 volatile 对象赋值给一个非 volatile 对象
    Foo& operator=(volatile const Foo&);
    // 将一个 volatile 对象赋值给一个 volatile 对象
    Foo& operator=(volatile const Foo&) volatile;
    // Foo 类的剩余部分
};

```

尽管我们可以为 `volatile` 对象定义拷贝和赋值操作，但是一个更深层次的问题是拷贝 `volatile` 对象是否有意义呢？不同程序使用 `volatile` 的目的各不相同，对上述问题的回答与具体的使用目的密切相关。

19.8.3 链接指示：extern "C"

C++ 程序有时需要调用其他语言编写的函数，最常见的是调用 C 语言编写的函数。像所有其他名字一样，其他语言中的函数名字也必须在 C++ 中进行声明，并且该声明必须指定返回类型和形参列表。对于其他语言编写的函数来说，编译器检查其调用的方式与处理普通 C++ 函数的方式相同，但是生成的代码有所区别。C++ 使用链接指示（linkage directive）指出任意非 C++ 函数所用的语言。

Note

要想把 C++ 代码和其他语言（包括 C 语言）编写的代码放在一起使用，要求我们必须有权访问该语言的编译器，并且这个编译器与当前的 C++ 编译器是兼容的。

声明一个非 C++ 的函数

链接指示可以有两种形式：单个的或复合的。链接指示不能出现在类定义或函数定义的内部。同样的链接指示必须在函数的每个声明中都出现。

举个例子，接下来的声明显示了 `cstring` 头文件的某些 C 函数是如何声明的：

```
// 可能出现在 C++ 头文件 <cstring> 中的链接指示
// 单语句链接指示
extern "C" size_t strlen(const char *);
// 复合语句链接指示
extern "C" {
    int strcmp(const char*, const char*);
    char *strcat(char*, const char*);
}
```

链接指示的第一种形式包含一个关键字 `extern`，后面是一个字符串字面值常量以及一个“普通的”函数声明。

其中的字符串字面值常量指出了编写函数所用的语言。编译器应该支持对 C 语言的链接指示。此外，编译器也可能会支持其他语言的链接指示，如 `extern "Ada"`、`extern "FORTRAN"` 等。

链接指示与头文件

我们可以令链接指示后面跟上花括号括起来的若干函数的声明，从而一次性建立多个链接。花括号的作用是将适用于该链接指示的多个声明聚合在一起，否则花括号就会被忽略，花括号中声明的函数名字就是可见的，就好像在花括号之外声明的一样。

多重声明的形式可以应用于整个头文件。例如，C++ 的 `cstring` 头文件可能形如：

```
// 复合语句链接指示
extern "C" {
    #include <string.h>           // 操作 C 风格字符串的 C 函数
}
```

当一个 `#include` 指示被放置在复合链接指示的花括号中时，头文件中的所有普通函数声明都被认为是由链接指示的语言编写的。链接指示可以嵌套，因此如果头文件包含自带链接指示的函数，则该函数的链接不受影响。



C++ 从 C 语言继承的标准库函数可以定义成 C 函数，但并非必须：决定使用 C 还是 C++ 实现 C 标准库，是每个 C++ 实现的事情。

指向 extern "C" 函数的指针

编写函数所用的语言是函数类型的一部分。因此，对于使用链接指示定义的函数来说，它的每个声明都必须使用相同的链接指示。而且，指向其他语言编写的函数的指针必须与函数本身使用相同的链接指示：

```
// pf 指向一个 C 函数，该函数接受一个 int 返回 void
extern "C" void (*pf)(int);
```

当我们使用 `pf` 调用函数时，编译器认定当前调用的是一个 C 函数。

指向 C 函数的指针与指向 C++ 函数的指针是不一样的类型。一个指向 C 函数的指针

不能用在执行初始化或赋值操作后指向 C++ 函数，反之亦然。就像其他类型不匹配的问题一样，如果我们试图在两个链接指示不同的指针之间进行赋值操作，则程序将发生错误：

```
void (*pf1)(int);           // 指向一个 C++ 函数
extern "C" void (*pf2)(int); // 指向一个 C 函数
pf1 = pf2;                  // 错误：pf1 和 pf2 的类型不同
```



有的 C++ 编译器会接受之前的这种赋值操作并将其作为对语言的扩展，尽管从严格意义上来看它是非法的。

链接指示对整个声明都有效

当我们使用链接指示时，它不仅对函数有效，而且对作为返回类型或形参类型的函数指针也有效：

```
// f1 是一个 C 函数，它的形参是一个指向 C 函数的指针
extern "C" void f1(void(*) (int));
```

这条声明语句指出 f1 是一个不返回任何值的 C 函数。它有一个类型是函数指针的形参，其中的函数接受一个 int 形参返回为空。这个链接指示不仅对 f1 有效，对函数指针同样有效。当我们调用 f1 时，必须传给它一个 C 函数的名字或者指向 C 函数的指针。

因为链接指示同时作用于声明语句中的所有函数，所以如果我们希望给 C++ 函数传入一个指向 C 函数的指针，则必须使用类型别名（参见 2.5.1 节，第 60 页）：

860

```
// FC 是一个指向 C 函数的指针
extern "C" typedef void FC(int);
// f2 是一个 C++ 函数，该函数的形参是指向 C 函数的指针
void f2(FC *);
```

导出 C++ 函数到其他语言

通过使用链接指示对函数进行定义，我们可以令一个 C++ 函数在其他语言编写的程序中可用：

```
// calc 函数可以被 C 程序调用
extern "C" double calc(double dparm) { /* ... */ }
```

编译器将为该函数生成适合于指定语言的代码。

值得注意的是，可被多种语言共享的函数的返回类型或形参类型受到很多限制。例如，我们不太可能把一个 C++ 类的对象传给 C 程序，因为 C 程序根本无法理解构造函数、析构函数以及其他类特有的操作。

对链接到 C 的预处理器的支持

有时需要在 C 和 C++ 中编译同一个源文件，为了实现这一目的，在编译 C++ 版本的程序时预处理器定义 `__cplusplus`（两个下画线）。利用这个变量，我们可以在编译 C++ 程序的时候有条件地包含进来一些代码：

```
#ifdef __cplusplus
// 正确：我们正在编译 C++ 程序
extern "C"
#endif
int strcmp(const char*, const char*);
```

重载函数与链接指示

链接指示与重载函数的相互作用依赖于目标语言。如果目标语言支持重载函数，则为该语言实现链接指示的编译器很可能也支持重载这些 C++ 的函数。

C 语言不支持函数重载，因此也就不难理解为什么一个 C 链接指示只能用于说明一组重载函数中的某一个了：

```
// 错误：两个 extern "C" 函数的名字相同
extern "C" void print(const char*);
extern "C" void print(int);
```

如果在组重载函数中有一个是 C 函数，则其余的必定都是 C++ 函数：

```
class SmallInt { /* ... */ };
class BigNum { /* ... */ };
// C 函数可以在 C 或 C++ 程序中调用
// C++ 函数重载了该函数，可以在 C++ 程序中调用
extern "C" double calc(double);
extern SmallInt calc(const SmallInt&);
extern BigNum calc(const BigNum&);
```

861

C 版本的 calc 函数可以在 C 或 C++ 程序中调用，而使用了类类型形参的 C++ 函数只能在 C++ 程序中调用。上述性质与声明的顺序无关。

19.8.3 节练习

练习 19.26：说明下列声明语句的含义并判断它们是否合法：

```
extern "C" int compute(int *, int);
extern "C" double compute(double *, double);
```

小结

C++为解决某些特殊问题设置了一系列特殊的处理机制。

有的程序需要精确控制内存分配过程，它们可以通过在类的内部或在全局作用域中自定义 `operator new` 和 `operator delete` 来实现这一目的。如果应用程序为这两个操作定义了自己的版本，则 `new` 和 `delete` 表达式将优先使用应用程序定义的版本。

有的程序需要在运行时直接获取对象的动态类型，运行时类型识别 (RTTI) 为这种程序提供了语言级别的支持。RTTI 只对定义了虚函数的类有效；对没有定义虚函数的类，虽然也可以得到其类型信息，但只是静态类型。

当我们定义指向类成员的指针时，在指针类型中包含了该指针所指成员所属类的类型信息。成员指针可以绑定到该类当中任意一个具有指定类型的成员上。当我们解引用成员指针时，必须提供获取成员所需的对象。

C++定义了另外几种聚集类型：

- 嵌套类，定义在其他类的作用域中，嵌套类通常作为外层类的实现类。
- `union`，是一种特殊的类，它可以定义几个数据成员但是在任意时刻只有一个成员有值，`union` 通常嵌套在其他类的内部。
- 局部类，定义在函数的内部，局部类的所有成员都必须定义在类内，局部类不能含有静态数据成员。

C++支持几种固有的不可移植的特性，其中位域和 `volatile` 使得程序更容易访问硬件；链接指示使得程序更容易访问用其他语言编写的代码。

术语表

匿名 union (anonymous union) 未命名的 `union`，不能用于定义对象。匿名 `union` 的成员也是外层作用域的成员。匿名 `union` 不能包含成员函数，也不能包含私有成员或受保护的成员。

位域 (bit-field) 特殊的类成员，该成员含有一个整型值以指定为其分配的二进制位数。如果可能的话，在类中连续定义的位域将被压缩在一个普通的整数值当中。

判别式 (discriminant) 是一种使用一个对象判断 `union` 的当前值类型的编程技术。

dynamic_cast 是一个运算符，执行从基类向派生类的带检查的强制类型转换。当基类中至少含有一个虚函数时，该运算符负责检查指针或引用所绑定的对象的动态类型。如果对象类型与目标类型（或其派生类）一致，则类型转换完成。否则，指针

转换将返回一个值为 0 的指针；引用转换将抛出一个异常。

枚举类型 (enumeration) 将一组整型常量命名后聚合在一起形成的类型。

枚举成员 (enumerator) 是枚举类型的成员。枚举成员是常量，可以用在任何需要整型常量的地方。

free 是定义在 `cstdlib` 中的低层函数，负责释放内存。`free` 只能释放由 `malloc` 分配的内存。

链接指示 (linkage directive) 支持 C++ 程序调用其他语言编写的函数的一种机制。所有编译器都应支持调用 C++ 和 C 函数，至于是否支持其他语言则由编译器决定。

局部类 (local class) 定义在函数中的类。局部类只有在其外层函数内可见。局部类